

LAPORAN PRAKTIKUM
INTERNET OF THINGS II

Fondasi Arsitektur Internet of Things (IoT) dan Mikrokontroler



Di susun oleh:

ANDREAN FIRNANDO (25782090)

TRI – 3D

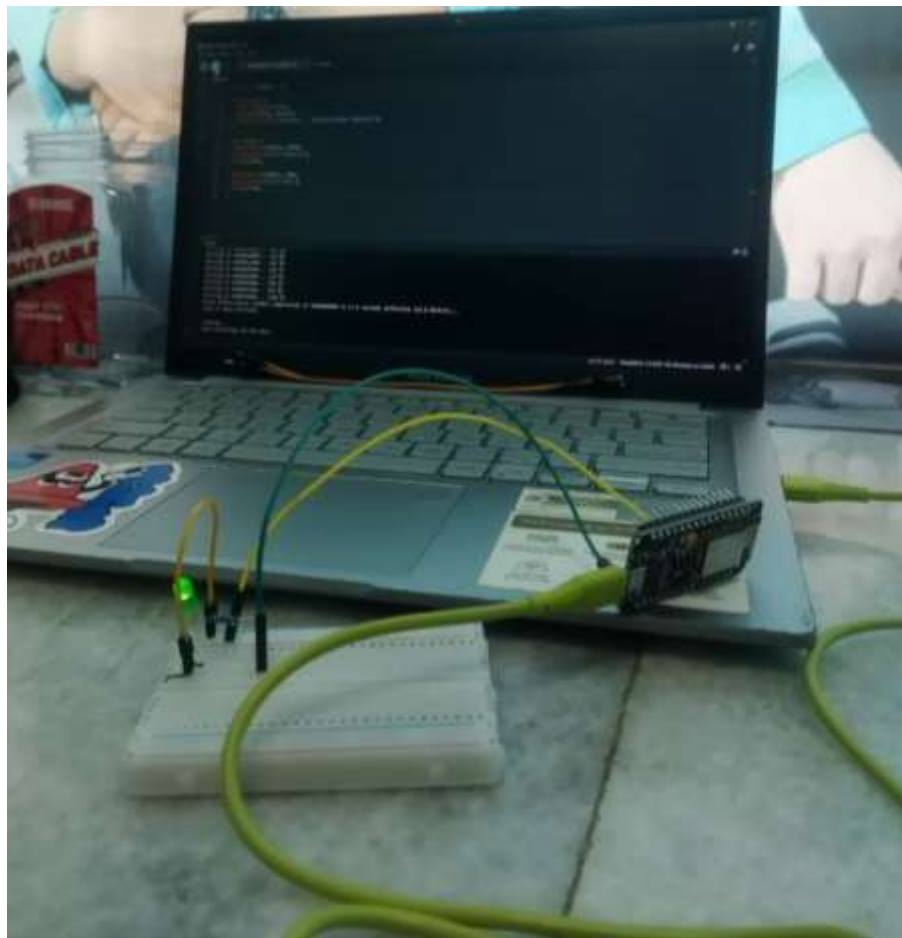
Dosen Pengajar :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M. T. Agus

Ambarwari, S.Pd., M.kom

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027

Praktikum 1



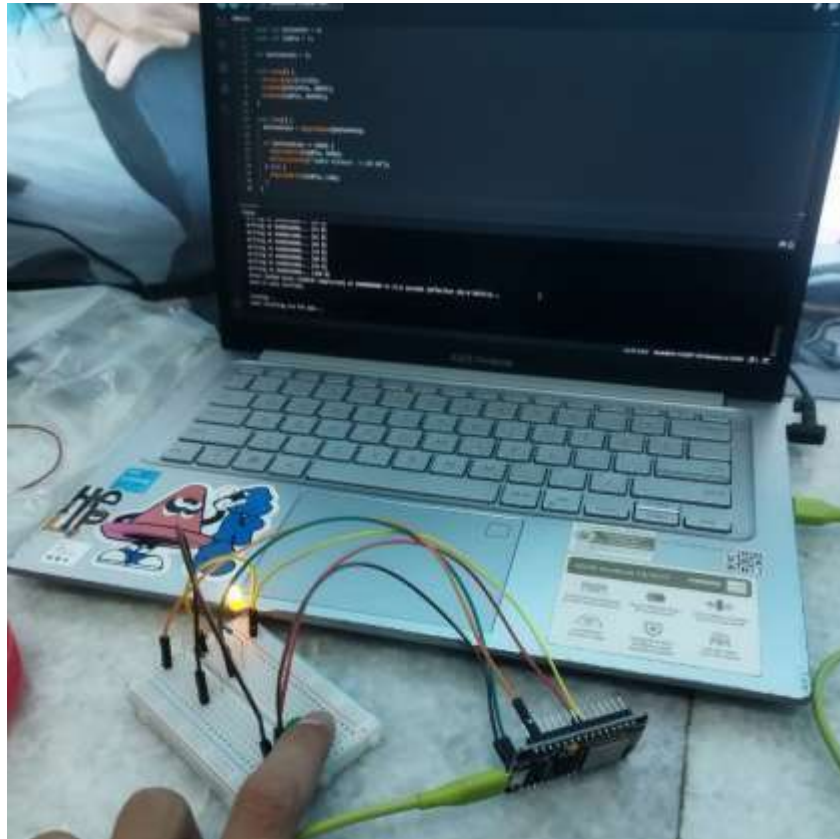
The screenshot shows the Arduino IDE interface. The main editor window contains a C++ sketch for a NodeMCU ESP-12 module. The sketch defines two digital pins, LED_PIN (5) and BEEP_PIN (4), and uses digitalWrite and digitalWriteFast functions to control an LED and a buzzer. The output window shows the serial monitor displaying '123 Muzika' and '123 Muzi'.

```

1 // NodeMCU ESP-12
2
3 #define LED_PIN 5
4 #define BEEP_PIN 4
5
6 void setup() {
7   Serial.begin(115200);
8   pinMode(LED_PIN, OUTPUT);
9   Serial.println("Fastlane 1 - digital output channel");
10 }
11
12 void loop() {
13   digitalWrite(LED_PIN, HIGH);
14   Serial.println("123 Muzika");
15   delay(1000);
16
17   digitalWrite(LED_PIN, LOW);
18   Serial.println("123 Muzi");
19   delay(1000);
20 }
  
```

The output window shows the serial monitor displaying '123 Muzika' and '123 Muzi'.

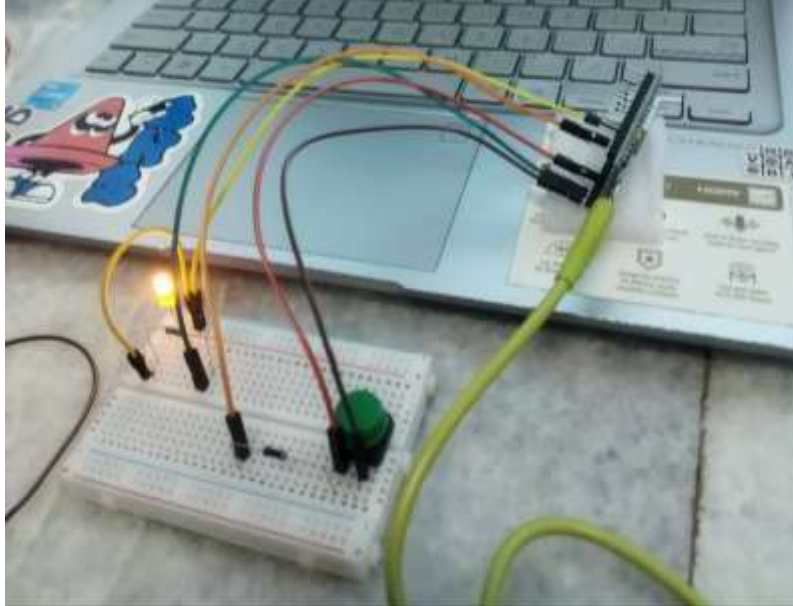
Praktikum 2



```
Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino 1.0.0 (SP-10-M...)
Sketch
1 const int buttonPin = 4;
2 const int ledPin = 13;
3
4 int buttonState = 0;
5
6 void setup() {
7   Serial.begin(115200);
8   pinMode(buttonPin, INPUT);
9   pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13   buttonState = digitalRead(buttonPin);
14
15   if (buttonState == HIGH) {
16     digitalWrite(ledPin, HIGH);
17     Serial.println("Button pressed -> LED ON");
18   } else {
19     digitalWrite(ledPin, LOW);
20   }
21 }
22
Output
Writing to EEPROM... (0x 0)
Writing at 0x00000000... (20 B)
Writing at 0x00000000... (64 B)
Writing at 0x00020000... (60 B)
Writing at 0x00030000... (76 B)
Writing at 0x00030000... (48 B)
Writing at 0x00030000... (92 B)
Writing at 0x00030000... (100 B)
Write 265984 bytes (104530 compressed) at 0x00000000 in 17.8 seconds (effective 122.8 kbit/s)...
Hash of data verified...
Leaving...
Hard resetting via DTR pin...
In 2.3.10 - Arduino 1.0.0 (SP-10-M...) Sketchbook (C:\Users\...)
```

- **Latihan 1 (Pemahaman Logika Pin):**

Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

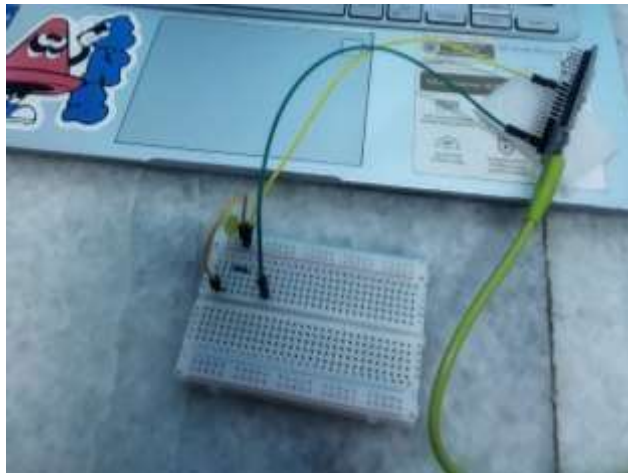


Memindahkan kabel pin D2 ke pin D8 (GPIO 15) dan menekannya saat *booting* akan mengunci sistem ESP8266 pada mode bootloader. Akibatnya, program utama mati total dan aktuator/LED terkunci pada kondisi menyala terus (atau mati terus) tanpa bisa dikendalikan.

- **Latihan 2 (Implementasi Kode):**

Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.





- `delay(200);`
- `delay(800);` pada latihan ini hanya mendelay mati nyala lampu tidak mempengaruhi yg lain.

• Latihan 3 (Analisis Sirkuit):

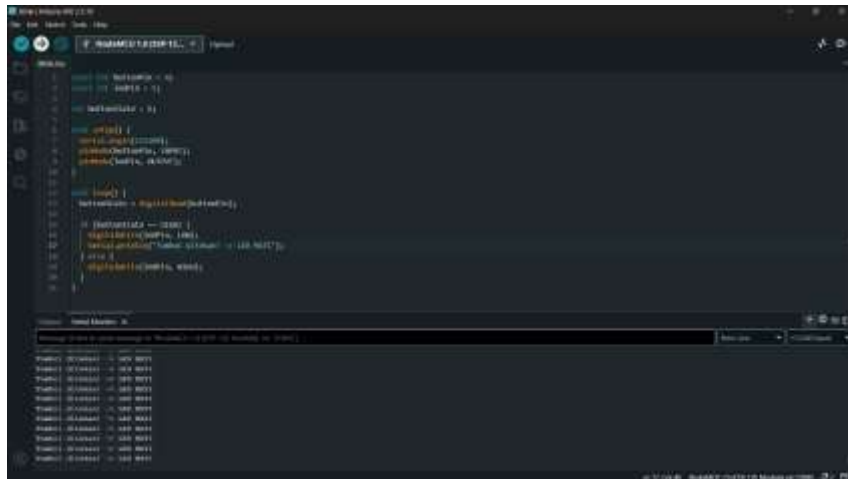
Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?



Program tidak mengalami error, dan tetap sukses dapat di unduh ke mikrokontroler,

LED tidak menyala, karena komponen pada kondisi ini, dioda akan menyumbat/menahan aliran listrik sehingga tidak ada arus yang mengalir, yang mengakibatkan LED tidak memancarkan cahaya sama sekali.

- **Latihan 4 (Pengembangan Algoritma):** Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru **mati**."

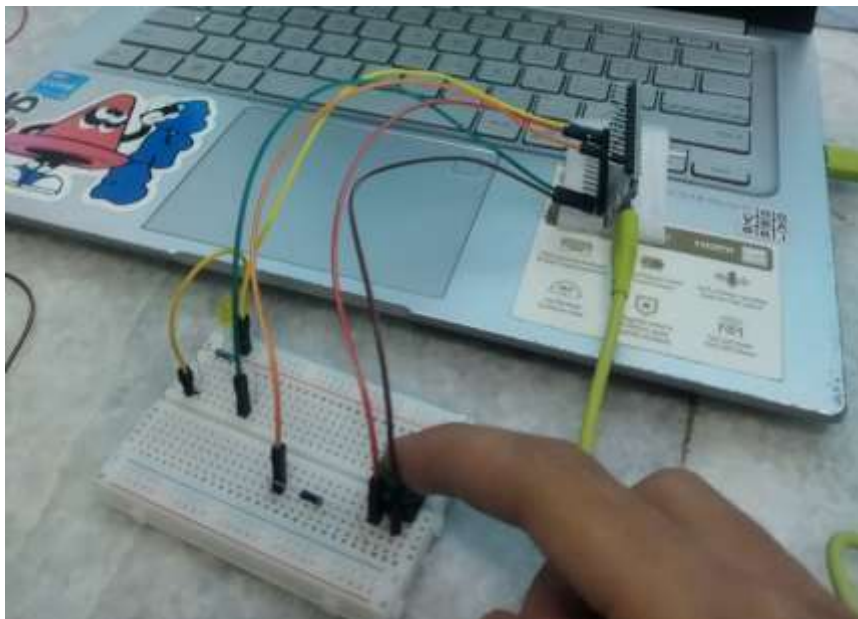


```
int ledPin = 13; // LED pin
int buttonPin = 2; // Button pin

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

void loop() {
  int buttonState = digitalRead(buttonPin);

  if (buttonState == LOW) {
    digitalWrite(ledPin, HIGH); // LED ON
  } else {
    digitalWrite(ledPin, LOW); // LED OFF
  }
}
```

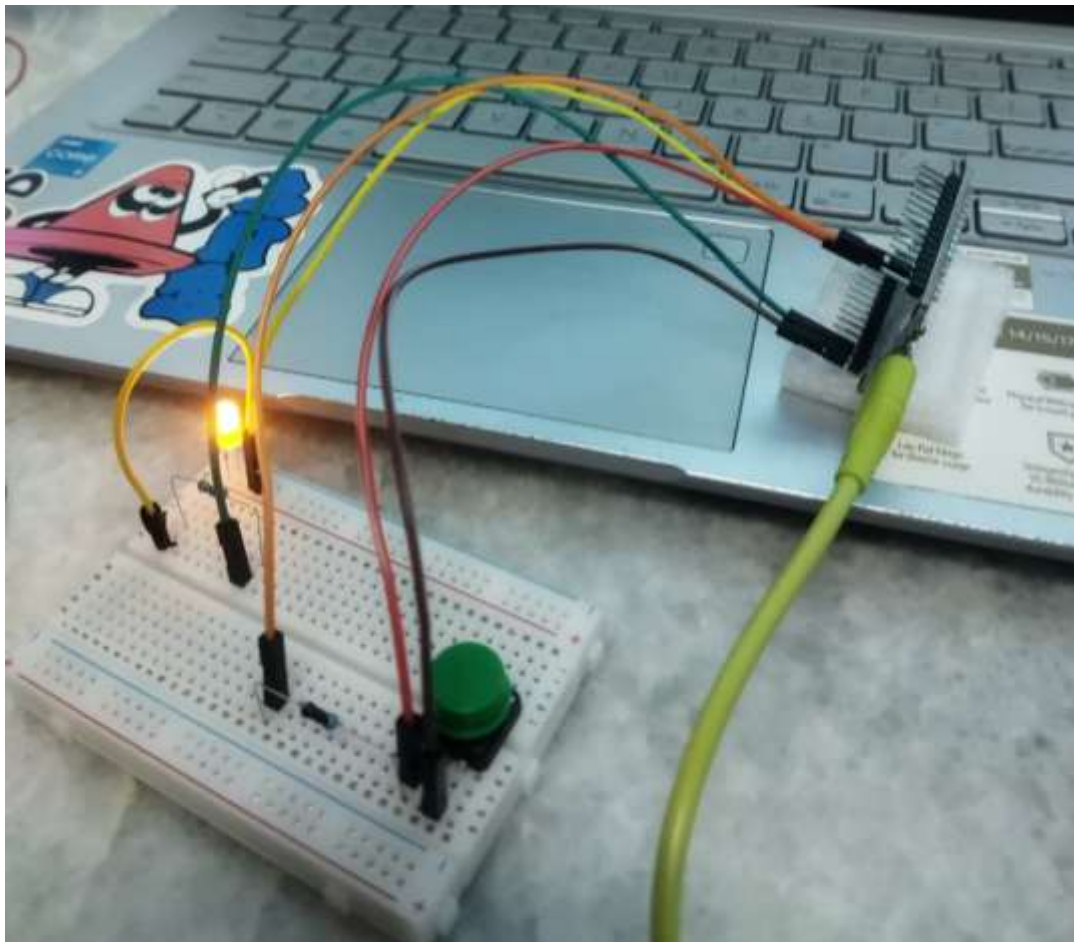


Setelah melakukan percobaan menggunakan simulasi wokwi, secara konsep saya mengerti, dan paham kondisi ini bisa berjalan, dengan menggunakan kondisi if-else, dan hanya bertukar posisi saat distart LED langsung menyala dan button di tekan maka mati, jadi kondisi ini hanya membalikan logika nya saja.

1.7 Tugas

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdint.h>
4  #include <stdlib.h>
5  #include <math.h>
6
7  typedef struct {
8      uint32_t H[4];
9      uint32_t E[4];
10     uint32_t S[4];
11     uint32_t W[4];
12     uint32_t T[4];
13     uint32_t U[4];
14     uint32_t V[4];
15     uint32_t X[4];
16     uint32_t Y[4];
17     uint32_t Z[4];
18     uint32_t A[4];
19     uint32_t B[4];
20     uint32_t C[4];
21     uint32_t D[4];
22     uint32_t E[4];
23     uint32_t F[4];
24     uint32_t G[4];
25     uint32_t H[4];
26     uint32_t I[4];
27     uint32_t J[4];
28     uint32_t K[4];
29     uint32_t L[4];
30     uint32_t M[4];
31     uint32_t N[4];
32     uint32_t O[4];
33     uint32_t P[4];
34     uint32_t Q[4];
35     uint32_t R[4];
36     uint32_t S[4];
37     uint32_t T[4];
38     uint32_t U[4];
39     uint32_t V[4];
40     uint32_t W[4];
41     uint32_t X[4];
42     uint32_t Y[4];
43     uint32_t Z[4];
44 } SHA256_CTX;
45
46 void SHA256_Init(SHA256_CTX *ctx) {
47     ctx->H[0] = 0x6A09E667;
48     ctx->H[1] = 0xBB67AE85;
49     ctx->H[2] = 0x3C62F6E0;
50     ctx->H[3] = 0xA54F492B;
51     ctx->E[0] = 0x00000000;
52     ctx->E[1] = 0x00000000;
53     ctx->E[2] = 0x00000000;
54     ctx->E[3] = 0x00000000;
55     ctx->S[0] = 0x00000000;
56     ctx->S[1] = 0x00000000;
57     ctx->S[2] = 0x00000000;
58     ctx->S[3] = 0x00000000;
59     ctx->W[0] = 0x00000000;
60     ctx->W[1] = 0x00000000;
61     ctx->W[2] = 0x00000000;
62     ctx->W[3] = 0x00000000;
63     ctx->T[0] = 0x00000000;
64     ctx->T[1] = 0x00000000;
65     ctx->T[2] = 0x00000000;
66     ctx->T[3] = 0x00000000;
67     ctx->U[0] = 0x00000000;
68     ctx->U[1] = 0x00000000;
69     ctx->U[2] = 0x00000000;
70     ctx->U[3] = 0x00000000;
71     ctx->V[0] = 0x00000000;
72     ctx->V[1] = 0x00000000;
73     ctx->V[2] = 0x00000000;
74     ctx->V[3] = 0x00000000;
75     ctx->X[0] = 0x00000000;
76     ctx->X[1] = 0x00000000;
77     ctx->X[2] = 0x00000000;
78     ctx->X[3] = 0x00000000;
79     ctx->Y[0] = 0x00000000;
80     ctx->Y[1] = 0x00000000;
81     ctx->Y[2] = 0x00000000;
82     ctx->Y[3] = 0x00000000;
83     ctx->Z[0] = 0x00000000;
84     ctx->Z[1] = 0x00000000;
85     ctx->Z[2] = 0x00000000;
86     ctx->Z[3] = 0x00000000;
87     ctx->A[0] = 0x00000000;
88     ctx->A[1] = 0x00000000;
89     ctx->A[2] = 0x00000000;
90     ctx->A[3] = 0x00000000;
91     ctx->B[0] = 0x00000000;
92     ctx->B[1] = 0x00000000;
93     ctx->B[2] = 0x00000000;
94     ctx->B[3] = 0x00000000;
95     ctx->C[0] = 0x00000000;
96     ctx->C[1] = 0x00000000;
97     ctx->C[2] = 0x00000000;
98     ctx->C[3] = 0x00000000;
99     ctx->D[0] = 0x00000000;
100    ctx->D[1] = 0x00000000;
101    ctx->D[2] = 0x00000000;
102    ctx->D[3] = 0x00000000;
103    ctx->I[0] = 0x00000000;
104    ctx->I[1] = 0x00000000;
105    ctx->I[2] = 0x00000000;
106    ctx->I[3] = 0x00000000;
107    ctx->J[0] = 0x00000000;
108    ctx->J[1] = 0x00000000;
109    ctx->J[2] = 0x00000000;
110    ctx->J[3] = 0x00000000;
111    ctx->K[0] = 0x00000000;
112    ctx->K[1] = 0x00000000;
113    ctx->K[2] = 0x00000000;
114    ctx->K[3] = 0x00000000;
115    ctx->L[0] = 0x00000000;
116    ctx->L[1] = 0x00000000;
117    ctx->L[2] = 0x00000000;
118    ctx->L[3] = 0x00000000;
119    ctx->M[0] = 0x00000000;
120    ctx->M[1] = 0x00000000;
121    ctx->M[2] = 0x00000000;
122    ctx->M[3] = 0x00000000;
123    ctx->N[0] = 0x00000000;
124    ctx->N[1] = 0x00000000;
125    ctx->N[2] = 0x00000000;
126    ctx->N[3] = 0x00000000;
127    ctx->O[0] = 0x00000000;
128    ctx->O[1] = 0x00000000;
129    ctx->O[2] = 0x00000000;
130    ctx->O[3] = 0x00000000;
131    ctx->P[0] = 0x00000000;
132    ctx->P[1] = 0x00000000;
133    ctx->P[2] = 0x00000000;
134    ctx->P[3] = 0x00000000;
135    ctx->Q[0] = 0x00000000;
136    ctx->Q[1] = 0x00000000;
137    ctx->Q[2] = 0x00000000;
138    ctx->Q[3] = 0x00000000;
139    ctx->R[0] = 0x00000000;
140    ctx->R[1] = 0x00000000;
141    ctx->R[2] = 0x00000000;
142    ctx->R[3] = 0x00000000;
143    ctx->S[0] = 0x00000000;
144    ctx->S[1] = 0x00000000;
145    ctx->S[2] = 0x00000000;
146    ctx->S[3] = 0x00000000;
147    ctx->T[0] = 0x00000000;
148    ctx->T[1] = 0x00000000;
149    ctx->T[2] = 0x00000000;
150    ctx->T[3] = 0x00000000;
151    ctx->U[0] = 0x00000000;
152    ctx->U[1] = 0x00000000;
153    ctx->U[2] = 0x00000000;
154    ctx->U[3] = 0x00000000;
155    ctx->V[0] = 0x00000000;
156    ctx->V[1] = 0x00000000;
157    ctx->V[2] = 0x00000000;
158    ctx->V[3] = 0x00000000;
159    ctx->W[0] = 0x00000000;
160    ctx->W[1] = 0x00000000;
161    ctx->W[2] = 0x00000000;
162    ctx->W[3] = 0x00000000;
163    ctx->X[0] = 0x00000000;
164    ctx->X[1] = 0x00000000;
165    ctx->X[2] = 0x00000000;
166    ctx->X[3] = 0x00000000;
167    ctx->Y[0] = 0x00000000;
168    ctx->Y[1] = 0x00000000;
169    ctx->Y[2] = 0x00000000;
170    ctx->Y[3] = 0x00000000;
171    ctx->Z[0] = 0x00000000;
172    ctx->Z[1] = 0x00000000;
173    ctx->Z[2] = 0x00000000;
174    ctx->Z[3] = 0x00000000;
175    ctx->A[0] = 0x00000000;
176    ctx->A[1] = 0x00000000;
177    ctx->A[2] = 0x00000000;
178    ctx->A[3] = 0x00000000;
179    ctx->B[
```



Pada praktikum ini kondisi awal saat sistem pertama kali dinyalakan, LED diatur agar berada dalam keadaan mati. Melalui penerapan teknik *state tracking*, program mampu membedakan perilaku tombol *momentary* biasa dengan sakelar *latching*. Logika ini memanfaatkan sejenis menyimpan kondisi aktif atau nonaktifnya lampu, serta menggunakan deteksi transisi sinyal dari kondisi LOW ke HIGH agar status LED hanya berubah ketika terjadi satu kali dorongan tombol.